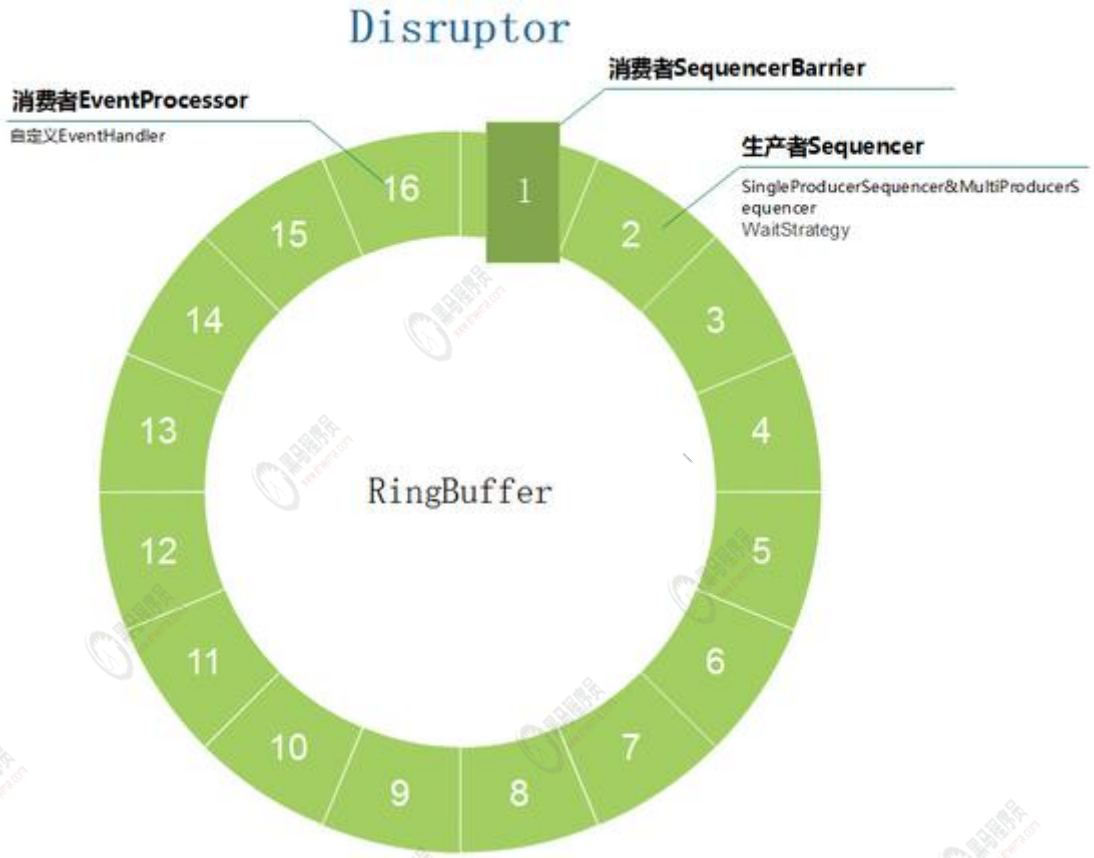
并发框架Disruptor



Disruptor概述

背景

Disruptor是英国外汇交易公司LMAX开发的一个高性能队列，研发的初衷是解决内存队列的延迟问题（在性能测试中发现竟然与I/O操作处于同样的数量级），基于Disruptor开发的系统单线程能支撑每秒600万订单，2010年在QCon演讲后，获得了业界关注，2011年，企业应用软件专家Martin Fowler专门撰写长文介绍。同年它还获得了Oracle官方的Duke大奖。

目前，包括Apache Storm、Camel、Log4j 2在内的很多知名项目都应用了Disruptor以获取高性能。

需要特别指出的是，这里所说的队列是系统内部的内存队列，而不是Kafka这样的分布式队列。

什么是Disruptor

Disruptor是用于一个JVM中多个线程之间的消息队列，作用与ArrayBlockingQueue有相似之处，但是Disruptor从功能、性能都远好于ArrayBlockingQueue，当多个线程之间传递大量数据或对性能要求较高时，可以考虑使用Disruptor作为ArrayBlockingQueue的替代者。

官方也对Disruptor和ArrayBlockingQueue的性能在不同的应用场景下做了对比，目测性能只有有5~10倍左右的提升。

为什么使用Disruptor

传统阻塞的队列使用锁保证线程安全，而锁通过操作系统内核上下文切换实现，会暂停线程去等待锁，直到锁释放。

执行这样的上下文切换，会丢失之前保存的数据和指令。由于消费者和生产者之间的速度差异，队列总是接近满或者空的状态，这种状态会导致高水平的写入争用。

传统队列问题

> 首先这里说的队列也仅限于Java内部的消息队列

队列	有界性	锁	结构	队列类型
ArrayBlockingQueue	有界	加锁	数组	阻塞
LinkedBlockingQueue	可选	加锁	链表	阻塞
ConcurrentLinkedQueue	无界	无锁	链表	非阻塞
LinkedTransferQueue	无界	无锁	链表	阻塞
PriorityBlockingQueue	无界	加锁	堆	阻塞
DelayQueue	无界	加锁	堆	阻塞

Disruptor应用场景

> 参考使用到disruptor的一些框架.

log4j2

Log4j2异步日志使用到了disruptor, 日志一般是有缓冲区, 满了才写到文件, 增量追加文件结合NIO等应该也比较快， 所以无论是EventHandler还是WorkHandler处理应该延迟比较小的， 写的文件也不多， 所以场景是比较合适的。

Jstorm

在流处理中不同线程中数据交换，数据计算可能蛮多内存中计算， 流计算快进快出，disruptor应该不错的选择。

百度uid-generator

部分使用ring buffer和去伪共享等思路缓存已生成的uid, 应该也部分参考了disruptor吧。

Disruptor 的核心概念

> 先从了解 Disruptor 的核心概念开始，来了解它是如何运作的。下面介绍的概念模型，既是领域对象，也是映射到代码实现上的核心对象。

Ring Buffer

> Disruptor中的数据结构，用于存储生产者生产的数据

如其名，环形的缓冲区。曾经 RingBuffer 是 Disruptor 中的最主要的对象，但从3.0版本开始，其职责被简化为仅仅负责对通过 Disruptor 进行交换的数据（事件）进行存储和更新。在一些更高级的应用场景中，Ring Buffer 可以由用户的自定义实现来完全替代。

Sequence

> 序号，在Disruptor框架中，任何地方都有序号

生产者生产的数据放在RingBuffer中的哪个位置，消费者应该消费哪个位置的数据，RingBuffer中的某个位置的数据是什么，这些都是由这个序号来决定的。这个序号可以简单的理解为一个AtomicLong类型的变量。其使用了padding的方法去消除缓存的伪共享问题。

Sequencer

> 序号生成器，这个类主要是用来协调生产者的

在生产者生产数据的时候，Sequencer会产生一个可用的序号（Sequence），然后生产者就就知道数据放在环形队列的那个位置了。

Sequencer是Disruptor的真正核心，此接口有两个实现类 SingleProducerSequencer、MultiProducerSequencer ，它们定义在生产者和消费者之间快速、正确地传递数据的并发算法。

Sequence Barrier

> 序号屏障

我们都知道，消费者在消费数据的时候，需要知道消费哪个位置的数据。消费者总不能自己想取哪个数据消费，就取哪个数据消费吧。这个SequencerBarrier起到的就是这样一个“栅栏”般的阻隔作用。你消费者想消费数据，得，我告诉你一个序号（Sequence），你去消费那个位置上的数据。要是没有数据，就好好等着吧

Wait Strategy

> Wait Strategy决定了一个消费者怎么等待生产者将事件（Event）放入Disruptor中。

设想一种这样的情景：生产者生产的非常慢，而消费者消费的非常快。那么必然会出现数据不够的情况，这个时候消费者怎么进行等待呢？WaitStrategy就是为了解决问题而诞生的。

Event

从生产者到消费者传递的数据叫做Event。它不是一个被 Disruptor 定义的特定类型，而是由 Disruptor 的使用者定义并指定。

EventHandler

Disruptor 定义的事件处理接口，由用户实现，用于处理事件，是 Consumer 的真正实现。

Producer

即生产者，只是泛指调用 Disruptor 发布事件的用户代码，Disruptor 没有定义特定接口或类型。

Disruptor特性

Disruptor其实就像一个队列一样，用于在不同的线程之间迁移数据，但是Disruptor也实现了一些其他队列没有的特性，如：

- 同一个“事件”可以有多个消费者，消费者之间既可以并行处理，也可以相互依赖形成处理的先后次序(形成一个依赖图)；
- 预分配用于存储事件内容的内存空间；
- 针对极高的性能目标而实现的极度优化和无锁的设计；

Disruptor入门

> 我们使用一个简单的例子来体验一下Disruptor，生产者会传递一个long类型的值到消费者，消费者接受到这个值后会打印出这个值。

添加依赖

```
<dependency>
    <groupId>com.lmax</groupId>
    <artifactId>disruptor</artifactId>
    <version>3.4.2</version>
</dependency>
```

Disruptor API

> Disruptor 的 API 十分简单，主要有以下几个步骤

定义事件

> 首先创建一个 `LongEvent` 类，这个类将会被放入环形队列中作为消息内容。
>
> 事件(Event)就是通过 Disruptor 进行交换的数据类型。

```
public class LongEvent {
    private long value;

    public void set(long value) {
        this.value = value;
    }

    public long getValue() {
        return value;
    }
}
```

定义事件工厂

> 为了使用Disruptor的内存预分配event，我们需要定义一个EventFactory

事件工厂(Event Factory)定义了如何实例化前面第1步中定义的事件(Event)，需要实现接口 com.lmax.disruptor.EventFactory<T>。

Disruptor 通过 EventFactory 在 RingBuffer 中预创建 Event 的实例。

一个 Event 实例实际上被用作一个“数据槽”，发布者发布前，先从 RingBuffer 获得一个 Event 的实例，然后往 Event 实例中填充数据，之后再发布到 RingBuffer 中，之后由 Consumer 获得该 Event 实例并从中读取数据。

```
public class LongEventFactory implements EventFactory<LongEvent> {
    public LongEvent newInstance() {
        return new LongEvent();
    }
}
```

定义事件处理的具体实现

> 为了让消费者处理这些事件，所以我们这里定义一个事件处理器，负责打印event
>
> 通过实现接口 com.lmax.disruptor.EventHandler<T> 定义事件处理的具体实现。

```
public class LongEventHandler implements EventHandler<LongEvent> {
    public void onEvent(LongEvent event, long sequence, boolean endOfBatch) {
        //CommonUtils.accumulation();
        System.out.println("consumer:" + Thread.currentThread().getName() + "
Event: value=" + event.getValue() + ",sequence=" + sequence);
    }
}
```

指定等待策略

> Disruptor 定义了 com.lmax.disruptor.WaitStrategy 接口用于抽象 Consumer 如何等待新事件，这是策略模式的应用

```
WaitStrategy YIELDING_WAIT = new YieldingWaitStrategy();
```

启动 Disruptor

> 注意ringBufferSize的大小必须是2的N次方

```
// 指定事件工厂
LongEventFactory factory = new LongEventFactory();

// 指定 ring buffer字节大小，必须是2的N次方
int bufferSize = 1024;

//单线程模式，获取额外的性能
Disruptor<LongEvent> disruptor = new Disruptor<LongEvent>(factory,
                                                          bufferSize,
Executors.defaultThreadFactory(),
                                                          ProducerType.SINGLE,
                                                          new
YieldingWaitStrategy());
//设置事件业务处理器---消费者
disruptor.handleEventsWith(new LongEventHandler());

//启动disruptor线程
disruptor.start();
```

使用Translators发布事件

> 在Disruptor的3.0版本中，由于加入了丰富的Lambda风格的API，可以用来帮组开发人员简化流程。所以在3.0版本后首选使用Event Publisher/Event Translator来发布事件。

```
public class LongEventProducerWithTranslator {
    private final RingBuffer<LongEvent> ringBuffer;

    public LongEventProducerWithTranslator(RingBuffer<LongEvent> ringBuffer) {
        this.ringBuffer = ringBuffer;
    }
```

```
    private static final EventTranslatorOneArg<LongEvent, Long> TRANSLATOR =
            new EventTranslatorOneArg<LongEvent, Long>() {
                public void translateTo(LongEvent event, long sequence, Long
data) {
                    event.set(data);
                }
            };

    public void onData(Long data) {
        ringBuffer.publishEvent(TRANSLATOR, data);
    }
}
```

关闭 Disruptor

```
disruptor.shutdown();//关闭 disruptor，方法会堵塞，直至所有的事件都得到处理
```

代码整合

LongEventMain

> 消费者-生产者启动类,其依靠构造Disruptor对象，调用start()方法完成启动线程。Disruptor 需要ringbuffer环，消费者数据处理工厂，WaitStrategy等

- ByteBuffer 类字节buffer，用于包装消息。
- ProducerType.SINGLE为单线程 ，可以提高性能

```
public class LongEventMain {
    public static void main(String[] args) {
        // 指定事件工厂
        LongEventFactory factory = new LongEventFactory();

        // 指定 ring buffer字节大小，必须是2的N次方
        int bufferSize = 1024;

        //单线程模式，获取额外的性能
        Disruptor<LongEvent> disruptor = new Disruptor<LongEvent>(factory,
                bufferSize, Executors.defaultThreadFactory(),
                ProducerType.SINGLE,
                new YieldingWaitStrategy());

        //设置事件业务处理器---消费者
        disruptor.handleEventsWith(new LongEventHandler());

        //启动disruptor线程
        disruptor.start();
        // 获取 ring buffer环，用于接取生产者生产的事件
        RingBuffer<LongEvent> ringBuffer = disruptor.getRingBuffer();

        //为 ring buffer指定事件生产者
        LongEventProducerWithTranslator producer = new
LongEventProducerWithTranslator(ringBuffer);
        //循环遍历
        for (int i = 0; i < 100; i++) {
            //获取一个随机数
            long value = (long) ((Math.random() * 1000000) + 1);
            //发布数据
```

```
            producer.onData(value);
        }
        //停止disruptor线程
        disruptor.shutdown();
    }
}
```

运行测试

> 测试结果

```
consumer:pool-1-thread-1 Event: value=579797,sequence=0
consumer:pool-1-thread-1 Event: value=974942,sequence=1
consumer:pool-1-thread-1 Event: value=978977,sequence=2
consumer:pool-1-thread-1 Event: value=398080,sequence=3
consumer:pool-1-thread-1 Event: value=867251,sequence=4
consumer:pool-1-thread-1 Event: value=796707,sequence=5
consumer:pool-1-thread-1 Event: value=786555,sequence=6
consumer:pool-1-thread-1 Event: value=182193,sequence=7
.....
```

> Event: value = 为消费者接收到的数据，sequence为数据在ringbuffer环的位置。

性能对比测试

> 为了直观地感受 Disruptor 有多快，设计了一个性能对比测试：Producer 发布 1 亿次事件，从发布第一个事件开始计时，捕捉 Consumer 处理完所有事件的耗时。
>
> 测试用例在 Producer 如何将事件通知到 Consumer 的实现方式上，设计了两种不同的实现：

2. Producer 的事件发布和 Consumer 的事件处理在不同的线程，通过 ArrayBlockingQueue 传递给 Consumer 进行处理；
3. Producer 的事件发布和 Consumer 的事件处理在不同的线程，通过 Disruptor 传递给 Consumer 进行处理；

代码实现

计算代码

> 进行CAS累加运算

```
public class CommonUtils {
    private static AtomicLong count = new AtomicLong(0);

    public static void calculation() {
        count.incrementAndGet();
    }

    public static long get() {
        return count.get();
    }
}
```

抽象类

> 进行一亿次 CAS运算计算耗时

```
/**
 * 抽象类
 *
 * @param <T>
 */
public abstract class AbstractTask<T> {
    private static final Logger logger =
LoggerFactory.getLogger(AbstractTask.class);
    //线程池
    private static final ExecutorService executor =
Executors.newFixedThreadPool(Runtime.getRuntime().availableProcessors() + 1);
    private static final CountDownLatch countDownLatch = new CountDownLatch(1);
    //一亿次测试
    public static long tasksize = 100000000;


    /**
     * 开始调用测试
     */
    public void invok() {
        //计算当前事件
        long currentTime = System.currentTimeMillis();
        //获取到监听器
        Runnable monitor = monitor();
        if (null != monitor) {
            executor.execute(monitor);
        }
        //启动
        start();

        //执行任务发布
        Runnable runnable = getTask();
        for (long i = 0; i < tasksize; i++) {
            runnable.run();
        }

        //停止任务
        stop();
        //等待任务发布完成
        try {
            countDownLatch.await();
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        executor.shutdown();
        //获取处理结果
        T result = getResult();
        //计算耗时
        long duration = System.currentTimeMillis() - currentTime;
        //计算吞吐量
        long throughput = (tasksize / duration) * 1000;
        logger.info("每秒吞吐量:[{}];({}/{})", throughput, result, duration);
    }
```

```java
    /**
     * 获取监听器
     *
     * @return
     */
    public Runnable monitor() {
        return null;
    }

    /**
     * 启动任务
     */
    public void start() {

    }

    /**
     * 完成任务
     */
    public void complete() {
        countDownLatch.countDown();
    }

    /**
     * 停止任务
     */
    public void stop() {

    }

    /**
     * 获取需要执行的任务
     *
     * @return
     */
    public abstract Runnable getTask();

    /**
     * 获取运行结果
     *
     * @return
     */
    public abstract T getResult();
}
```

Disruptor性能测试代码

```java
public class DisruptorTest extends AbstractTask<Long> {
    //定义随机数生成器
    private static final Random r = new Random();
    //定义Disruptor对象
    private Disruptor disruptor = null;
    //定义Disruptor事件发布对象
    private LongEventProducerWithTranslator translator = null;
```

```java
    /**
     * 启动
     */
    @Override
    public void start() {
        //定义事件工厂
        EventFactory<LongEvent> eventFactory = new LongEventFactory();
        // RingBuffer 大小，必须是 2 的 N 次方;
        int ringBufferSize = 1024 * 1024;
        //构建disruptor对象
        disruptor = new Disruptor<LongEvent>(eventFactory,
                ringBufferSize, Executors.defaultThreadFactory(),
ProducerType.SINGLE,
                new YieldingWaitStrategy());
        //定义事件处理类
        EventHandler<LongEvent> eventHandler = new LongEventHandler();
        //配置事件处理类
        disruptor.handleEventsWith(eventHandler);
        //启动disruptor
        disruptor.start();
        //创建事件发布对象
        translator = new
LongEventProducerWithTranslator(disruptor.getRingBuffer());
    }

    /**
     * 停止任务
     */
    @Override
    public void stop() {
        disruptor.shutdown();
        System.out.println("运算结果: " + CommonUtils.get());
        //完成任务
        complete();
    }

    /**
     * 获取需要执行的任务
     *
     * @return
     */
    @Override
    public Runnable getTask() {
        return () -> {
            publishEvent();
        };
    }

    /**
     * 获取运行结果
     *
     * @return
     */
    @Override
    public Long getResult() {
        return CommonUtils.get();
    }
```

```
    /**
     * 发布对象
     */
    private void publishEvent() {
        //获取要通过事件传递的业务数据
        Long data = r.nextLong();
        // 发布事件
        translator.onData(data);
    }


    public static void main(String[] args) {
        DisruptorTest disruptorTest = new DisruptorTest();
        disruptorTest.invok();
    }

}
```

> 输出结果

```
10:45:22.941 [main] INFO com.heima.task.AbstractTask - 每秒吞吐量:[18171000];
(100000000/5503)
```

ArrayBlockingQueue性能测试代码

```
public class ArrayBlockingQueueTest extends AbstractTask {
    private static final Random r = new Random();
    private static final ArrayBlockingQueue<Long> queue = new
ArrayBlockingQueue(10000000);


    @Override
    public Runnable monitor() {
        return () -> {
            try {
                for (int i = 0; i < tasksize; i++) {
                    //获取一个元素
                    queue.take();
                    //执行计算
                    CommonUtils.calculation();
                }
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
            complete();
        };
    }

    public static void main(String[] args) {
        ArrayBlockingQueueTest test = new ArrayBlockingQueueTest();
        test.invok();
    }

    @Override
```

```
    public Runnable getTask() {
        return () -> {
            publish();
        };
    }

    @Override
    public Object getResult() {
        return CommonUtils.get();
    }

    public void publish() {
        Long data = r.nextLong();
        try {
            queue.put(data);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
}
```

> 输出结果

```
10:45:46.379 [main] INFO com.heima.task.AbstractTask - 每秒吞吐量:[6192000];
(100000000/16148)
```

测试对比

测试类	运算次数	耗时(ms)	吞吐量/s
ArrayBlockingQueue	1亿次	16148	6192000
Disruptor	1亿次	5503	18171000

Disruptor官方性能测试

> Disruptor论文中讲述了一个实验：

- 这个测试程序调用了一个函数，该函数会对一个64位的计数器循环自增5亿次。
- 机器环境：2.4G 6核
- 运算： 64位的计数器累加5亿次

Method	Time (ms)
单线程	300
单线程使用 CAS	5,700
单线程使用锁	10,000
单线程使用volatile	4,700
多线程使用 CAS	30,000
多线程使用锁	224,000

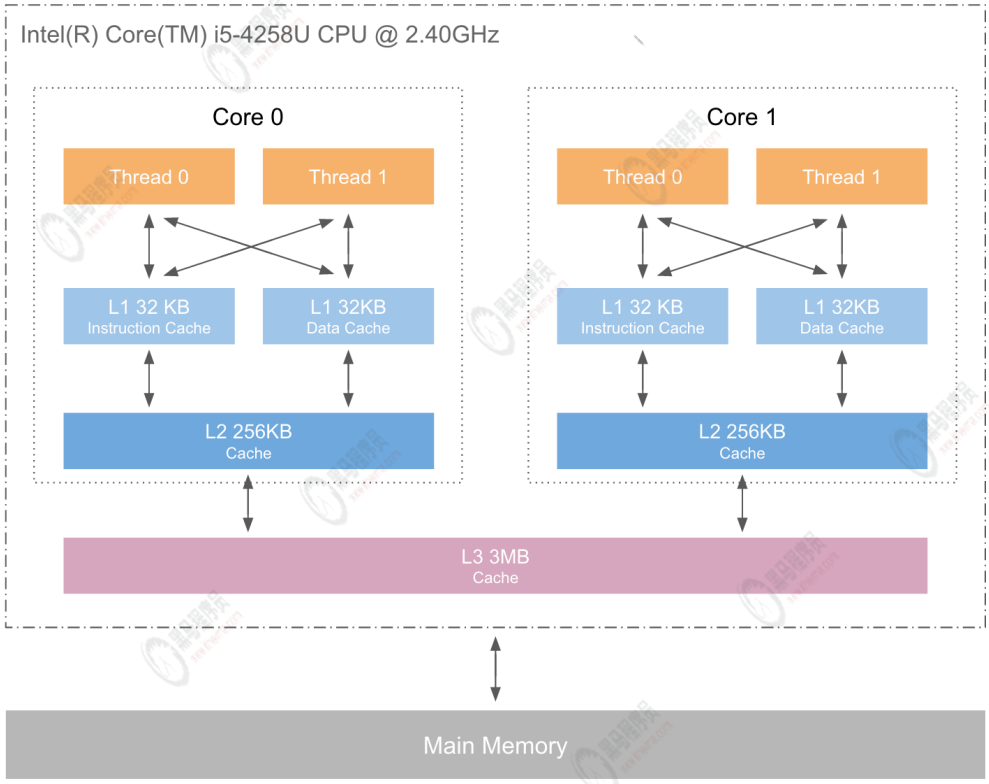
高性能原理

- 引入环形的数组结构：数组元素不会被回收，避免频繁的GC，
- 无锁的设计：采用CAS无锁方式，保证线程的安全性
- 属性填充：通过添加额外的无用信息，避免伪共享问题
- 元素位置的定位：采用跟一致性哈希一样的方式，一个索引，进行自增

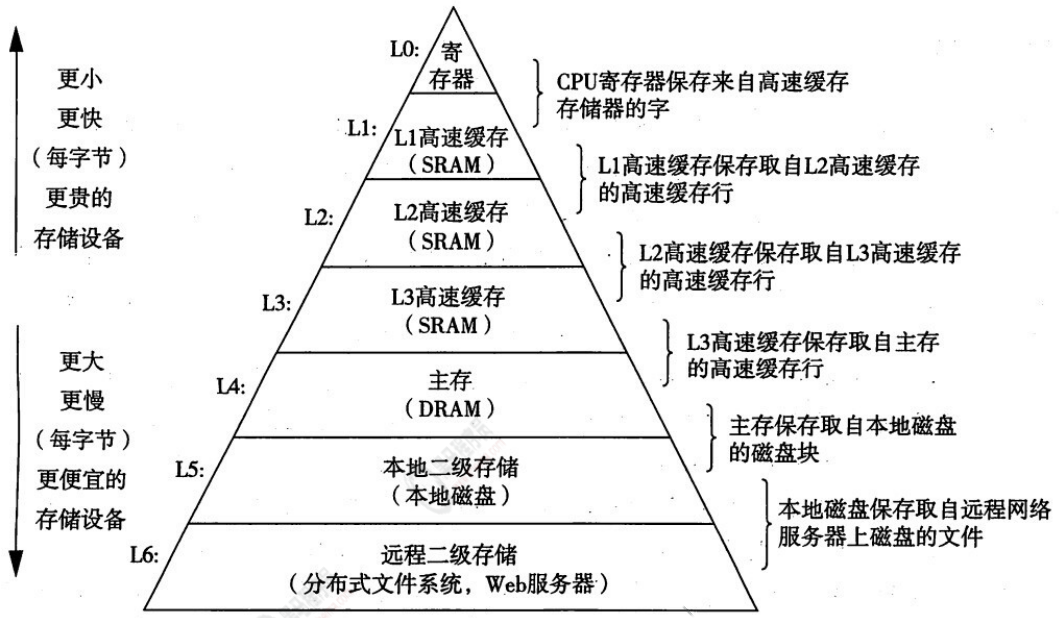
伪共享概念

计算机缓存构成

下图是计算的基本结构。L1、L2、L3分别表示一级缓存、二级缓存、三级缓存，越靠近CPU的缓存，速度越快，容量也越小，所以L1缓存很小但很快，并且紧靠着在使用它的CPU内核；L2大一些，也慢一些，并且仍然只能被一个单独的CPU核使用；L3更大、更慢，并且被单个插槽上的所有CPU核共享；最后是主存，由全部插槽上的所有CPU核共享。



当CPU要读取一个数据时，首先从一级缓存中查找，如果没有找到再从二级缓存中查找，如果还是没有就从三级缓存或内存中查找。一般来说，每级缓存的命中率大概都在80%左右，也就是说全部数据量的80%都可以在一级缓存中找到，只剩下20%的总数据量才需要从二级缓存、三级缓存或内存中读取，由此可见一级缓存是整个CPU缓存架构中最为重要的部分。



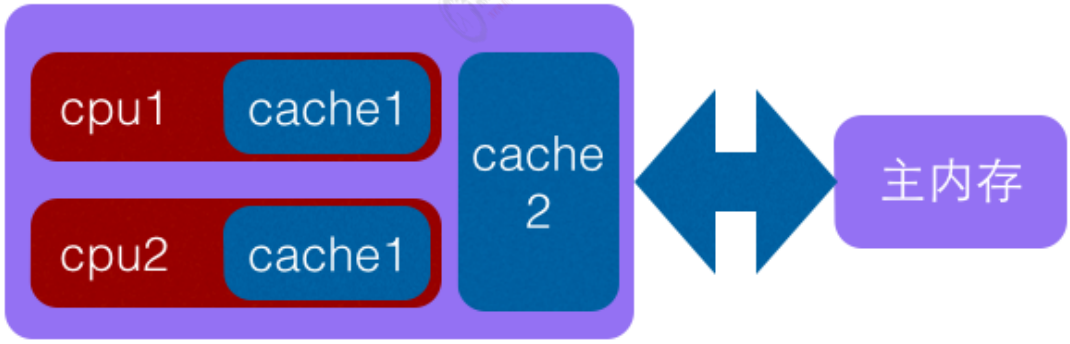
> 下表是一些缓存未命中的消耗数据：

从CPU到	大约需要的CPU周期	大约需要的时间
主存		约60-80ns
QPI总线		约20ns
L3 cache	约40-45cycles	约15ns
L2 cache	约10cycles	约3ns
L1 cache	约3-4cycles	约1ns
寄存器	1cycle	

> 可见CPU读取主存中的数据会比从L1中读取慢了近2个数量级。

什么是缓存行

为了解决计算机系统中主内存与 CPU 之间运行速度差问题，会在 CPU 与主内存之间 添加一级或者多级高速缓冲存储器(Cache)。这个 Cache 一般是被集成到 CPU 内部的， 所以也叫 CPU Cache，如图所示是两级 Cache 结构。



Cache内部是按行存储的，其中每一行称为一个cache line，由很多个 Cache line 组成的，Cache line 是 cache 和 RAM 交换数据的最小单位，cache行的大小一般为2的幂次数字节，通常为 64 Byte。Cache line是Cache与主内存进行数据交换的单位。

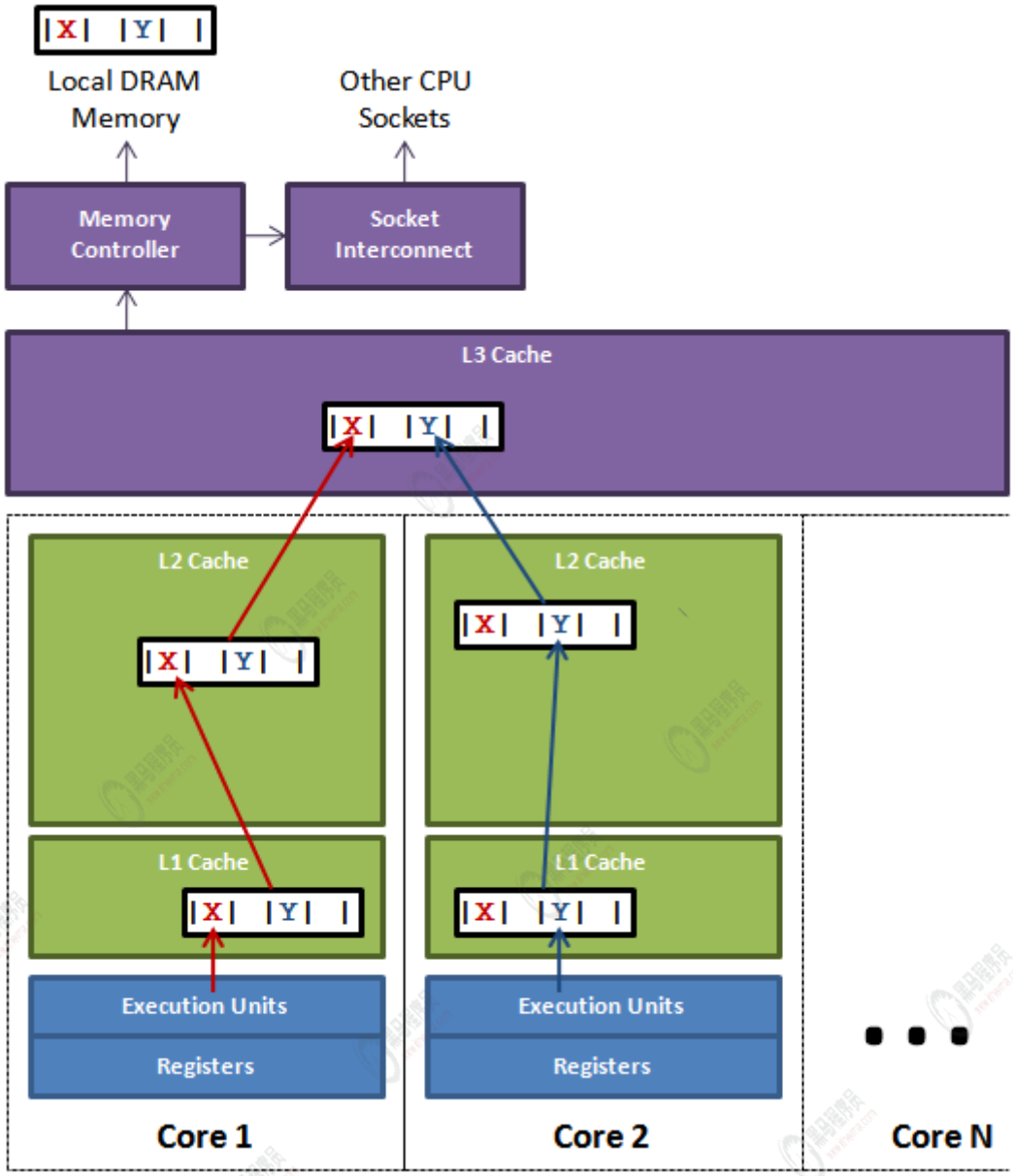
当 CPU 把内存的数据载入 cache 时，会把临近的共 64 Byte 的数据一同放入同一个Cache line，因为空间局部性：临近的数据在将来被访问的可能性大。

> linux 查看缓存行大小

```
more /sys/devices/system/cpu/cpu1/cache/index0/coherency_line_size
64
```

什么是共享

CPU缓存是以缓存行（cache line）为单位存储的。缓存行通常是 64 字节，并且它有效地引用主内存中的一块地址。一个 Java 的 long 类型是 8 字节，因此在一个缓存行中可以存 8 个 long 类型的变量。所以，如果你访问一个 long 数组，当数组中的一个值被加载到缓存中，它会额外加载另外 7 个，以致你能非常快地遍历这个数组。事实上，你可以非常快速的遍历在连续的内存块中分配的任意数据结构。而如果你在数据结构中的项在内存中不是彼此相邻的（如链表），你将得不到免费缓存加载所带来的优势，并且在这些数据结构中的每一个项都可能会出现缓存未命中。下图是一个CPU缓存行的示意图：

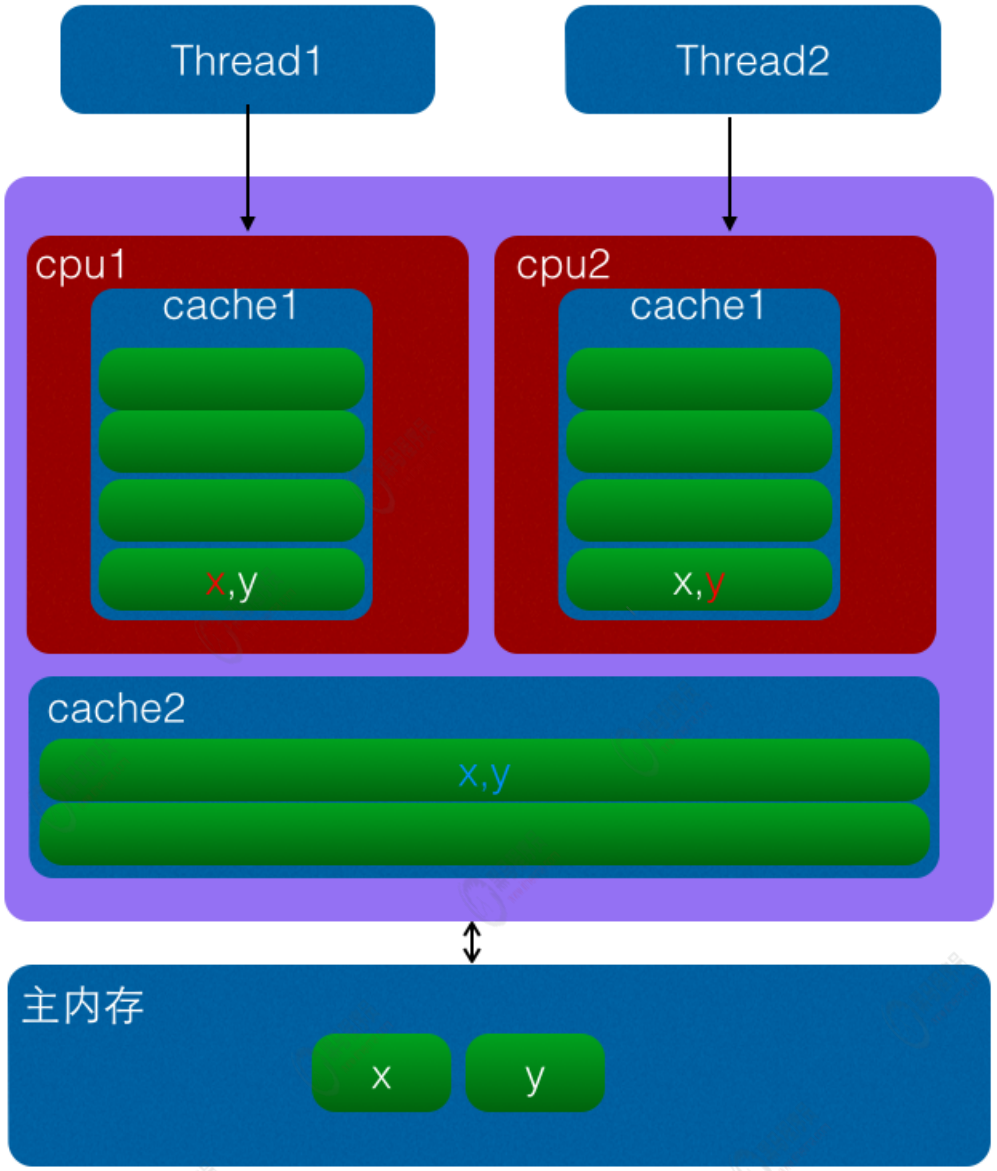


表面上 X 和 Y 都是被独立线程操作的，而且两操作之间也没有任何关系。只不过它们共享了一个缓存行，但所有竞争冲突都是来源于共享。

什么是伪共享

当CPU访问某一个变量时候，首先会去看CPU Cache内是否有该变量，如果有则直接从中获取，否者就去主内存里面获取该变量，然后把该变量所在内存区域的一个Cache行大小的内存拷贝到Cache（cache行是Cache与主内存进行数据交换的单位）。

由于存放到Cache行的的是内存块而不是单个变量，所以可能会把多个变量存放到了一个cache行。当多个线程同时修改一个缓存行里面的多个变量时候，由于同时只能有一个线程操作缓存行，所以相比每个变量放到一个缓存行性能会有所下降，这就是伪共享。



如上图变量x,y同时被放到了CPU的一级和二级缓存，当线程1使用CPU1对变量x进行更新时候，首先会修改cpu1的一级缓存变量x所在缓存行，这时候缓存一致性协议会导致cpu2中变量x对应的缓存行失效，那么线程2写入变量x的时候就只能去二级缓存去查找，这就破坏了一级缓存，而一级缓存比二级缓存更快。更坏的情况下如果cpu只有一级缓存，那么会导致频繁的直接访问主内存。

我们的缓存都是以缓存行作为一个单位来处理的，所以失效x的缓存的同时，也会把y失效，反之亦然。

为何会出现伪共享

伪共享的产生是因为多个变量被放入了一个缓存行，并且多个线程同时去写入缓存行中不同变量。那么为何多个变量会被放入一个缓存行那。其实是因为Cache与内存交换数据的单位就是Cache line，当CPU要访问的变量没有在Cache命中时候，根据程序运行的局部性原理会把该变量在内存中大小为Cache行的内存放如缓存行。

```
long a;
long b;
long c;
long d;
```

如上代码，声明了四个long变量，假设cache line的大小为32个字节，那么当cpu访问变量a时候发现该变量没有在cache命中，那么就会去主内存把变量a以及内存地址附近的b,c,d放入缓存行。也就是地址连续的多个变量才有可能会被放到一个缓存行中，当创建数组时候，数组里面的多个元素就会被放入到同一个缓存行。那么单线程下多个变量放入缓存行对性能有影响？其实正常情况下单线程访问时候由于数组元素被放入到了一个或者多个cache行对代码执行是有利的，因为数据都在缓存中，代码执行会更快。

如何解伪共享

解决伪共享最直接的方法就是填充（padding），例如下面的VolatileLong，一个long占8个字节，Java的对象头占用8个字节（32位系统）或者12字节（64位系统，默认开启对象头压缩，不开启占16字节）。一个缓存行64字节，那么我们可以填充6个long（6 * 8 = 48 个字节）。

不使用字段填充

```
public class VolatileData {
    // 占用 8个字节 +48 + 对象头 = 64字节

    //需要操作的数据
    volatile long value;

    public VolatileData() {
    }

    public VolatileData(long defValue) {
        value = defValue;
    }

    public long accumulationAdd() {
        //因为单线程操作不需要加锁
        value++;
        return value;
    }

    public long getValue() {
        return value;
    }
}
```

> 内存布局

```
-------------------------------
com.heima.cacheline.VolatileData object internals:
 OFFSET  SIZE   TYPE DESCRIPTION                               VALUE
      0     4        (object header)                           01 00 00 00 (00000001 00000000 00000000 00000000) (1)
      4     4        (object header)                           00 00 00 00 (00000000 00000000 00000000 00000000) (0)
      8     4        (object header)                           43 c1 00 f8 (01000011 11000001 00000000 11111000) (-134168253)
     12     4        (alignment/padding gap)
     16     8   long VolatileData.value                        0
Instance size: 24 bytes
Space losses: 4 bytes internal + 0 bytes external = 4 bytes total
-------------------------------
com.heima.cacheline.VolatileData@e73f9acd object externals:
          ADDRESS       SIZE TYPE                             PATH                           VALUE
        76be98190         24 com.heima.cacheline.VolatileData                                (object)

-------------------------------
size : 24
```

填充字段

> 因为JDK1.7以后就自动优化代码会删除无用的代码，在JDK1.7以后的版本这些不生效了。

```
/**
 * 缓存行填充父类
 */
public class DataPadding {
    //填充 5个long类型字段 8*5 = 40 个字节
    private long p1, p2, p3, p4, p5;
    //需要操作的数据
    volatile long value;
}
```

> 内存布局

```
com.heima.cacheline.VolatileData object internals:
 OFFSET  SIZE   TYPE DESCRIPTION                               VALUE
      0     4        (object header)                           01 00 00 00 (00000001 00000000 00000000 00000000) (1)
      4     4        (object header)                           00 00 00 00 (00000000 00000000 00000000 00000000) (0)
      8     4        (object header)                           43 c1 00 f8 (01000011 11000001 00000000 11111000) (-134168253)
     12     4        (alignment/padding gap)
     16     8   long VolatileData.p1                            0
     24     8   long VolatileData.p2                            0
     32     8   long VolatileData.p3                            0
     40     8   long VolatileData.p4                            0
     48     8   long VolatileData.p5                            0
     56     8   long VolatileData.value                         0
Instance size: 64 bytes
Space losses: 4 bytes internal + 0 bytes external = 4 bytes total
----------------------------------
com.heima.cacheline.VolatileData@e73f9acd object externals:
          ADDRESS       SIZE TYPE                             PATH                           VALUE
         76be98210         64 com.heima.cacheline.VolatileData                              (object)

----------------------------------
size : 64
```

继承的方式

```
/**
 * 缓存行填充父类
 */
public class DataPadding {
    //填充 5个long类型字段 8*5 = 40 个字节
    private long p1, p2, p3, p4, p5;
}
```

继承缓存填充类

```
/**
 * 继承DataPadding
 */
public class VolatileData extends DataPadding {
    // 占用 8个字节 +48 + 对象头 = 64字节

    public VolatileData() {
    }

    public VolatileData(long defValue) {
        value = defValue;
    }

    public long accumulationAdd() {
        //因为单线程操作不需要加锁
```

```
        value++;
        return value;
    }

    public long getValue() {
        return value;
    }
}
```

> 内存布局

```
-----------------------------------
com.heima.cacheline.VolatileData object internals:
 OFFSET  SIZE   TYPE DESCRIPTION                               VALUE
      0     4        (object header)                           01 00 00 00 (00000001 00000000 00000000 00000000) (1)
      4     4        (object header)                           00 00 00 00 (00000000 00000000 00000000 00000000) (0)
      8     4        (object header)                           81 c1 00 f8 (10000001 11000001 00000000 11111000) (-134168191)
     12     4        (alignment/padding gap)
     16     8   long DataPadding.p1                             0
     24     8   long DataPadding.p2                             0
     32     8   long DataPadding.p3                             0
     40     8   long DataPadding.p4                             0
     48     8   long DataPadding.p5                             0
     56     8   long DataPadding.value                          0
Instance size: 64 bytes
Space losses: 4 bytes internal + 0 bytes external = 4 bytes total
-----------------------------------
com.heima.cacheline.VolatileData@61064425d object externals:
          ADDRESS       SIZE TYPE                             PATH                           VALUE
         76be9a620         64 com.heima.cacheline.VolatileData                              (object)

-----------------------------------
size : 64
```

Disruptor填充方式

```
class LhsPadding {
    protected long p1, p2, p3, p4, p5, p6, p7;
}

class Value extends LhsPadding {
    protected volatile long value;
}

class RhsPadding extends Value {
    protected long p9, p10, p11, p12, p13, p14, p15;
}
```

继承填充类

```
public class VolatileData extends RhsPadding {
    // 占用 8个字节 +48 + 对象头 = 64字节

    //需要操作的数据
    volatile long value;

    public VolatileData() {
    }

    public VolatileData(long defValue) {
        value = defValue;
    }

    public long accumulationAdd() {
        //因为单线程操作不需要加锁
```

```
        value++;
        return value;
    }

    public long getValue() {
        return value;
    }
}
```

> 内存布局

```
---------------------------------
com.heima.cacheline.VolatileData object internals:
 OFFSET  SIZE   TYPE DESCRIPTION                               VALUE
      0     4        (object header)                           01 00 00 00 (00000001 00000000 00000000 00
      4     4        (object header)                           00 00 00 00 (00000000 00000000 00000000 00
      8     4        (object header)                           05 c2 00 f8 (00000101 11000010 00000000 11
     12     4        (alignment/padding gap)
     16     8   long LhsPadding.p1                              0
     24     8   long LhsPadding.p2                              0
     32     8   long LhsPadding.p3                              0
     40     8   long LhsPadding.p4                              0
     48     8   long LhsPadding.p5                              0
     56     8   long LhsPadding.p6                              0
     64     8   long LhsPadding.p7                              0
     72     8   long Value.value                                0
     80     8   long RhsPadding.p9                              0
     88     8   long RhsPadding.p10                             0
     96     8   long RhsPadding.p11                             0
    104     8   long RhsPadding.p12                             0
    112     8   long RhsPadding.p13                             0
    120     8   long RhsPadding.p14                             0
    128     8   long RhsPadding.p15                             0
    136     8   long VolatileData.value                         0
Instance size: 144 bytes
Space losses: 4 bytes internal + 0 bytes external = 4 bytes total
---------------------------------
com.heima.cacheline.VolatileData@299a06acd object externals:
          ADDRESS       SIZE TYPE                             PATH                           VALUE
        76be9e6a8        144 com.heima.cacheline.VolatileData                               (object)

---------------------------------
size : 144
```

@Contended注解

```
@Retention(RetentionPolicy.RUNTIME)
@Target({ElementType.FIELD, ElementType.TYPE})
public @interface Contended {
    String value() default "";
}
```

注解填充类

```
@Contended
public class VolatileData  {
    // 占用 8个字节 +48 + 对象头 = 64字节

    //需要操作的数据
    volatile long value;

    public VolatileData() {
    }
```

```java
    public VolatileData(long defValue) {
        value = defValue;
    }

    public long accumulationAdd() {
        //因为单线程操作不需要加锁
        value++;
        return value;
    }

    public long getValue() {
        return value;
    }
}
```

> 内存布局

```
--------------------------------
com.heima.cacheline.VolatileData object internals:
 OFFSET  SIZE   TYPE DESCRIPTION                               VALUE
      0     4        (object header)                           01 00 00 00 (00000001 00000000 00000000 00000000) (1)
      4     4        (object header)                           00 00 00 00 (00000000 00000000 00000000 00000000) (0)
      8     4        (object header)                           43 c1 00 f8 (01000011 11000001 00000000 11111000) (-134168253)
     12   132        (alignment/padding gap)
    144     8   long VolatileData.value                        0
    152   128        (loss due to the next object alignment)
Instance size: 280 bytes
Space losses: 132 bytes internal + 128 bytes external = 260 bytes total
--------------------------------
com.heima.cacheline.VolatileData@1055e4afd object externals:
          ADDRESS       SIZE TYPE                             PATH                           VALUE
         76be98210        280 com.heima.cacheline.VolatileData                                (object)

--------------------------------
size : 280
```

注意事项

在Java8中提供了**@sun.misc.Contended**来避免伪共享时，在运行时需要设置JVM启动参数**-XX:-RestrictContended**否则可能不生效。

性能对比

测试代码

> 使用和不使用缓存行填充的对比

```java
/**
 * 缓存行测试
 */
public class CacheLineTest {
    /**
     * 通过缓存行填充的变量
     */
    private VolatileData volatileData1 = new VolatileData(0);
    private VolatileData volatileData2 = new VolatileData(0);
    private VolatileData volatileData3 = new VolatileData(0);
    private VolatileData volatileData4 = new VolatileData(0);
    private VolatileData volatileData5 = new VolatileData(0);
    private VolatileData volatileData6 = new VolatileData(0);
    private VolatileData volatileData7 = new VolatileData(0);

    /**
     * 循环次数
     */
```

```java
    private final long size = 100000000;

    /**
     * 进行累加操作
     */
    public void accumulationX(VolatileData volatileData) {
        //计算耗时
        long currentTime = System.currentTimeMillis();
        long value = 0;
        //循环累加
        for (int i = 0; i < size; i++) {
            //使用缓存行填充的方式
            value = volatileData.accumulationAdd();


        }
        //打印
        System.out.println(value);
        //打印耗时
        System.out.println("耗时: " + (System.currentTimeMillis() -
currentTime));
    }


    public static void main(String[] args) {
        //创建对象
        CacheLineTest cacheRowTest = new CacheLineTest();
        //创建线程池
        ExecutorService executorService =
Executors.newFixedThreadPool(Runtime.getRuntime().availableProcessors());
        //启动三个线程个调用他们各自的方法
        executorService.execute(() ->
cacheRowTest.accumulationX(cacheRowTest.volatileData1));
        executorService.execute(() ->
cacheRowTest.accumulationX(cacheRowTest.volatileData2));
        executorService.execute(() ->
cacheRowTest.accumulationX(cacheRowTest.volatileData3));
        executorService.execute(() ->
cacheRowTest.accumulationX(cacheRowTest.volatileData4));
        executorService.execute(() ->
cacheRowTest.accumulationX(cacheRowTest.volatileData5));
        executorService.execute(() ->
cacheRowTest.accumulationX(cacheRowTest.volatileData6));
        executorService.execute(() ->
cacheRowTest.accumulationX(cacheRowTest.volatileData7));
        executorService.shutdown();
    }
}
```

测试数据

> 同样的结构他们之间差了 将近 50倍的速度差距

对象	NoPadding(MS)	DataPadding(MS)	RhsPadding(MS)	Contended(MS)
volatileData1	3751	1323	1307	1291
volatileData2	3790	1383	1311	1314
volatileData3	7551	1400	1311	1333
volatileData4	7669	1407	1317	1356
volatileData5	8577	1447	1327	1361
volatileData6	8705	1479	1339	1375
volatileData6	8741	1512	1368	1389

Disruptor解决伪共享

在Disruptor中有一个重要的类Sequence，该类包装了一个volatile修饰的long类型数据value，无论是Disruptor中的基于数组实现的缓冲区RingBuffer，还是生产者，消费者，都有各自独立的Sequence，RingBuffer缓冲区中，Sequence标示着写入进度，例如每次生产者要写入数据进缓冲区时，都要调用RingBuffer.next（）来获得下一个可使用的相对位置。对于生产者和消费者来说，Sequence标示着它们的事件序号，来看看Sequence类的源码：

```
class LhsPadding {
    protected long p1, p2, p3, p4, p5, p6, p7;
}

class Value extends LhsPadding {
    protected volatile long value;
}

class RhsPadding extends Value {
    protected long p9, p10, p11, p12, p13, p14, p15;
}

public class Sequence extends RhsPadding {
    static final long INITIAL_VALUE = -1L;
    private static final Unsafe UNSAFE;
    private static final long VALUE_OFFSET;
    static {
        UNSAFE = Util.getUnsafe();
        try {
            VALUE_OFFSET =
UNSAFE.objectFieldOffset(Value.class.getDeclaredField("value"));
        } catch(final Exception e) {
             throw new RuntimeException(e);
        }
    }



    public Sequence() {
        this(INITIAL_VALUE);
    }

    public Sequence(final long initialValue) {
        UNSAFE.putOrderedLong(this, VALUE_OFFSET, initialValue);
    }
```

```
}
```

从第1到11行可以看到，真正使用到的变量value，它的前后空间都由8个long型的变量填补了，对于一个大小为64字节的缓存行，它刚好被填补满（一个long型变量value，8个字节加上前/后个7long型变量填补，7*8=56，56+8=64字节）。这样做每次把变量value读进高速缓存中时，都能把缓存行填充满（对于大小为64个字节的缓存行来说，如果缓存行大小大于64个字节，那么还是会出现伪共享问题），保证每次处理数据时都不会与其他变量发生冲突。

无锁的设计

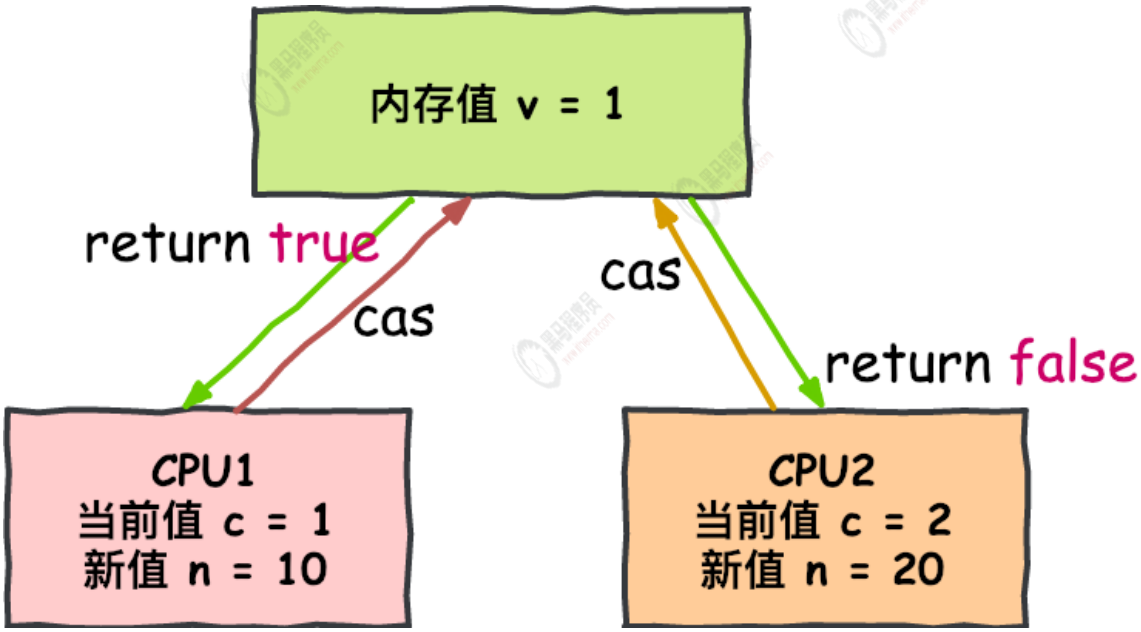
锁机制存在的问题

- 在多线程竞争下，加锁、释放锁会导致比较多的上下文切换和调度延时，引起性能问题，而且在上下文切换的时候，cpu之前缓存的指令和数据都将失效，对性能有很大的损失，用户态的锁虽然避免了这些问题，但是其实它们只是在没有真实的竞争时才有效。
- 一个线程持有锁会导致其它所有需要此锁的线程挂起直至该锁释放。
- 如果一个优先级高的线程等待一个优先级低的线程释放锁会导致导致优先级反转(Priority Inversion)，引起性能风险。

CAS无锁算法

实现无锁（lock-free）的非阻塞算法有多种实现方法，其中 CAS（比较与交换，Compare and swap）是一种有名的无锁算法。CAS的语义是“我认为V的值应该为A，如果是，那么将V的值更新为B，否则不修改并告诉V的值实际为多少”，CAS是一种 乐观锁 技术，当多个线程尝试使用CAS同时更新同一个变量时，只有其中一个线程能更新变量的值，而其它线程都失败，失败的线程并不会被挂起，而是被告知这次竞争中失败，并可以再次尝试。CAS有3个操作数，内存值V，旧的预期值A，要修改的新值B。当且仅当预期值A和内存值V相同时，将内存值V修改为B，否则什么都不做。

这是一个CPU级别的指令，在我的意识中，它的工作方式有点像乐观锁——CPU去更新一个值，但如果想改的值不再是原来的值，操作就失败，因为很明显，有其它操作先改变了这个值。



注意，这可以是CPU的两个不同的核心，但不会是两个独立的CPU。

CAS操作比锁消耗资源少的多，因为它们不牵涉操作系统，它们直接在CPU上操作。但它们并非没有代价——在上面的试验中，单线程无锁耗时300ms，单线程有锁耗时10000ms，单线程使用CAS耗时5700ms。所以它比使用锁耗时少，但比不需要考虑竞争的单线程耗时多。

传统队列问题

> 队列的底层数据结构一般分成三种：数组、链表和堆。其中，堆这里是为了实现带有优先级特性的队列，暂且不考虑。

队列	有界性	锁	数据结构
ArrayBlockingQueue	bounded	加锁	arraylist
LinkedBlockingQueue	optionally-bounded	加锁	linkedlist
ConcurrentLinkedQueue	unbounded	无锁	linkedlist
LinkedTransferQueue	unbounded	无锁	linkedlist
PriorityBlockingQueue	unbounded	加锁	heap
DelayQueue	unbounded	加锁	heap

在稳定性和性能要求特别高的系统中，为了防止生产者速度过快，导致内存溢出，只能选择有界队列；

同时，为了减少Java的垃圾回收对系统性能的影响，会尽量选择array/heap格式的数据结构。这样筛选下来，符合条件的队列就只有ArrayBlockingQueue，但是ArrayBlockingQueue是通过加锁的方式保证线程安全，而且ArrayBlockingQueue还存在伪共享问题，这两个问题严重影响了性能。

Disruptor的无锁设计

多线程环境下，多个生产者通过do/while循环的条件CAS，来判断每次申请的空间是否已经被其他生产者占据。假如已经被占据，该函数会返回失败，While循环重新执行，申请写入空间。

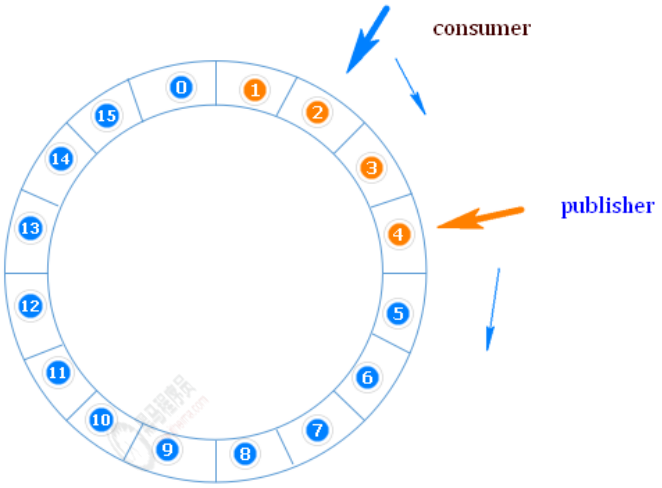
```
do
{
    current = cursor.get();
    next = current + n;

    if (!hasAvailableCapacity(gatingSequences, n, current))
    {
        throw InsufficientCapacityException.INSTANCE;
    }
}
while (!cursor.compareAndSet(current, next));
//next 类比于ArrayBlockQueue的数组索引index
return next;
```

环形数组结构

> 环形数组结构是整个Disruptor的核心所在。

什么是环形数组

RingBuffer 是一个环(首尾相连的环)，用做在不同上下文(线程)间传递数据的buffer，RingBuffer 拥有一个序号，这个序号指向数组中下一个可用元素。



为什么使用环形数组

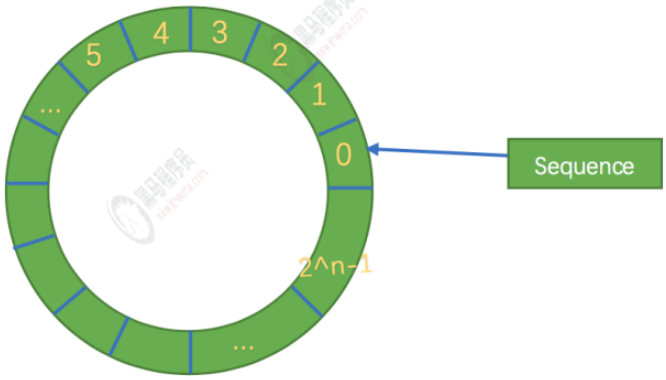
> 为了避免垃圾回收，采用数组而非链表。同时，数组对处理器的缓存机制更加友好

首先因为是数组，所以要比链表快，而且根据我们对上面缓存行的解释知道，数组中的一个元素加载，相邻的数组元素也是会被预加载的，因此在这样的结构中，cpu无需时不时去主存加载数组中的下一个元素。

而且，你可以为数组预先分配内存，使得数组对象一直存在（除非程序终止）。这就意味着不需要花大量的时间用于垃圾回收。

此外，不像链表那样，需要为每一个添加到其上面的对象创造节点对象—对应的，当删除节点时，需要执行相应的内存清理操作。环形数组中的元素采用覆盖方式，避免了jvm的GC。

其次结构作为环形，数组的大小为2的n次方，这样元素定位可以通过位运算效率会更高，这个跟一致性哈希中的环形策略有点像。在disruptor中，这个牛逼的环形结构就是RingBuffer，既然是数组，那么就有大小，而且这个大小必须是2的n次方，结构如下：



其实质只是一个普通的数组，只是当放置数据填充满队列（即到达2^n-1位置）之后，再填充数据，就会从0开始，覆盖之前的数据，于是就相当于一个环。

元素位置定位

数组长度`2^n`，通过位运算，加快定位的速度。下标采取递增的形式。不用担心index溢出的问题。index是long类型，即使100万QPS的处理速度，也需要30万年才能用完。

等待策略

定义 Consumer 如何进行等待下一个事件的策略。（注：Disruptor 定义了多种不同的策略，针对不同的场景，提供了不一样的性能表现）根据实际运行环境的 CPU 的硬件特点选择恰当的策略，并配合特定的 JVM 的配置参数，能够实现不同的性能提升。

BlockingWaitStrategy

Disruptor的默认策略是BlockingWaitStrategy，在BlockingWaitStrategy内部是使用锁和condition来控制线程的唤醒

BlockingWaitStrategy是最低效的策略，但其对CPU的消耗最小并且在各种不同部署环境中能提供更加一致的性能表现。

SleepingWaitStrategy

SleepingWaitStrategy 的性能表现跟 BlockingWaitStrategy 差不多，对 CPU 的消耗也类似，但其对生产者线程的影响最小，通过使用`LockSupport.parkNanos(1)`来实现循环等待，适合用于异步日志类似的场景；

YieldingWaitStrategy

YieldingWaitStrategy是可以使用在低延迟系统的策略之一，YieldingWaitStrategy将自旋以等待序列增加到适当的值。在循环体内，将调用`Thread.yield()`以允许其他排队的线程运行。在要求极高性能且事件处理线数小于 CPU 逻辑核心数的场景中，推荐使用此策略；

BusySpinWaitStrategy

性能最好，适合用于低延迟的系统。在要求极高性能且事件处理线程数小于CPU逻辑核心数的场景中，推荐使用此策略；

PhasedBackoffWaitStrategy

自旋 + yield + 自定义策略，CPU资源紧缺，吞吐量和延迟并不重要的场景。

生产和消费模式

> 根据上面的环形结构，我们来具体分析一下Disruptor的工作原理。

Disruptor 不像传统的队列，分为一个队头指针和一个队尾指针，而是只有一个角标（上面的seq），那么这个是如何保证生产的消息不会覆盖没有消费掉的消息呢。

在Disruptor中生产者分为单生产者和多生产者，而消费者并没有区分。

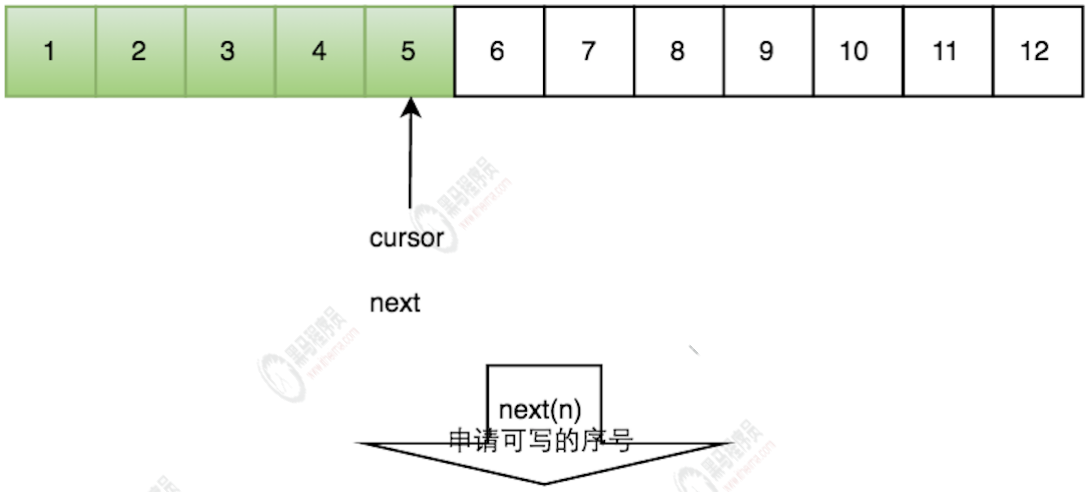
单生产者情况下，就是普通的生产者向RingBuffer中放置数据，消费者获取最大可消费的位置，并进行消费。而多生产者时候，又多出了一个跟RingBuffer同样大小的Buffer，称为AvailableBuffer。

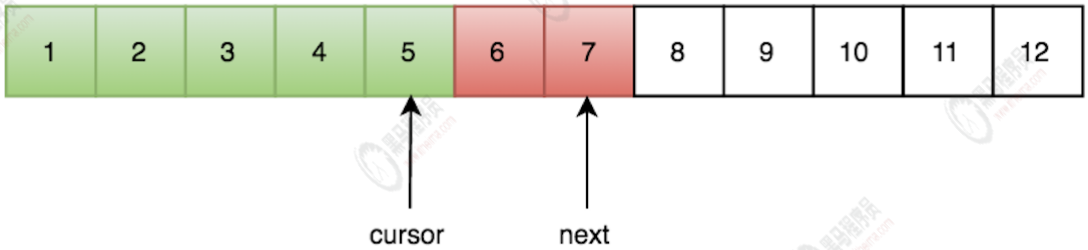
在多生产者中，每个生产者首先通过CAS竞争获取可以写的空间，然后再进行慢慢往里放数据，如果正好这个时候消费者要消费数据，那么每个消费者都需要获取最大可消费的下标，这个下标是在AvailableBuffer进行获取得到的最长连续的序列下标。

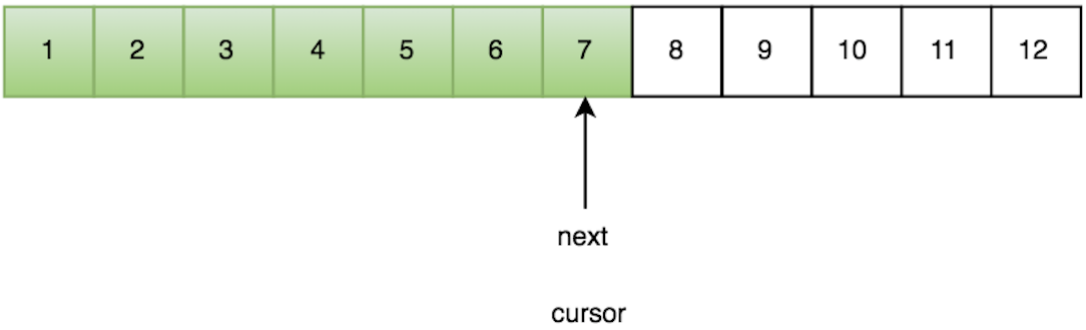
单生产者生产数据

> 生产者单线程写数据的流程比较简单：

1. 申请写入m个元素；
2. 若是有m个元素可以入，则返回最大的序列号。这儿主要判断是否会覆盖未读的元素；
3. 若是返回的正确，则生产者开始写入元素。







多生产者生产数据

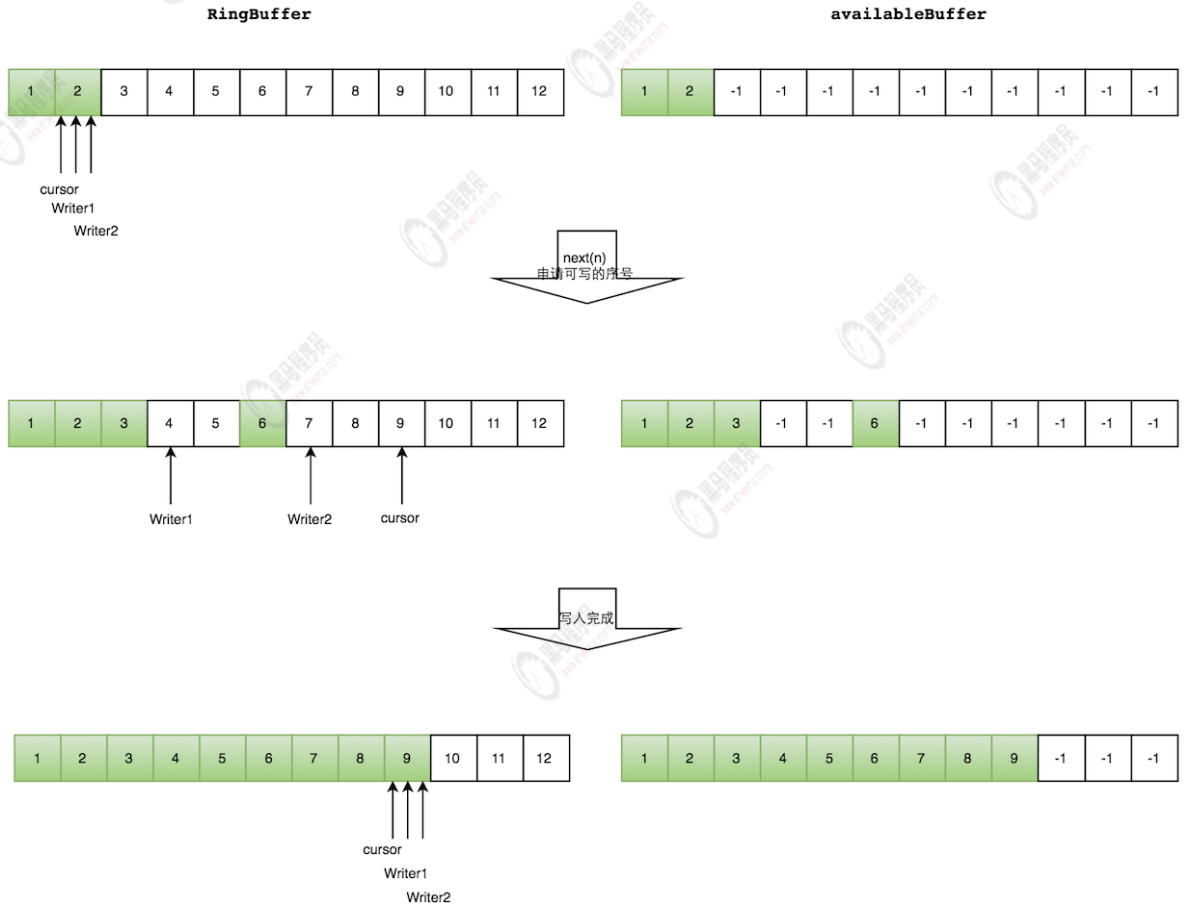
多个生产者的情况下，会遇到“如何防止多个线程重复写同一个元素”的问题。Disruptor的解决方法是，每个线程获取不同的一段数组空间进行操作。这个通过CAS很容易达到。只需要在分配元素的时候，通过CAS判断一下这段空间是否已经分配出去即可。

但是会遇到一个新问题：如何防止读取的时候，读到还未写的元素。Disruptor在多个生产者的情况下，引入了一个与Ring Buffer大小相同的buffer：available Buffer。当某个位置写入成功的时候，便把availble Buffer相应的位置置位，标记为写入成功。读取的时候，会遍历available Buffer，来判断元素是否已经就绪。

生产流程

1. 申请写入m个元素；
2. 若是有m个元素可以写入，则返回最大的序列号。每个生产者会被分配一段独享的空间；
3. 生产者写入元素，写入元素的同时设置available Buffer里面相应的位置，以标记自己哪些位置是已经写入成功的。

> 如下图所示，Writer1和Writer2两个线程写入数组，都申请可写的数组空间。Writer1被分配了下标3到下表5的空间，Writer2被分配了下标6到下标9的空间。
>
> Writer1写入下标3位置的元素，同时把available Buffer相应位置置位，标记已经写入成功，往后移一位，开始写下标4位置的元素。Writer2同样的方式。最终都写入完成。



CAS检测空间占用

> 防止不同生产者对同一段空间写入的代码，如下所示：

通过do/while循环的条件cursor.compareAndSet(current, next)，来判断每次申请的空间是否已经被其他生产者占据。假如已经被占据，该函数会返回失败，While循环重新执行，申请写入空间。

```
public long tryNext(int n) throws InsufficientCapacityException
{
    if (n < 1)
    {
        throw new IllegalArgumentException("n must be > 0");
    }

    long current;
    long next;

    do
    {
        current = cursor.get();
        next = current + n;

        if (!hasAvailableCapacity(gatingSequences, n, current))
        {
            throw InsufficientCapacityException.INSTANCE;
        }
    }
    while (!cursor.compareAndSet(current, next));

    return next;
}
```

多生产者消费数据

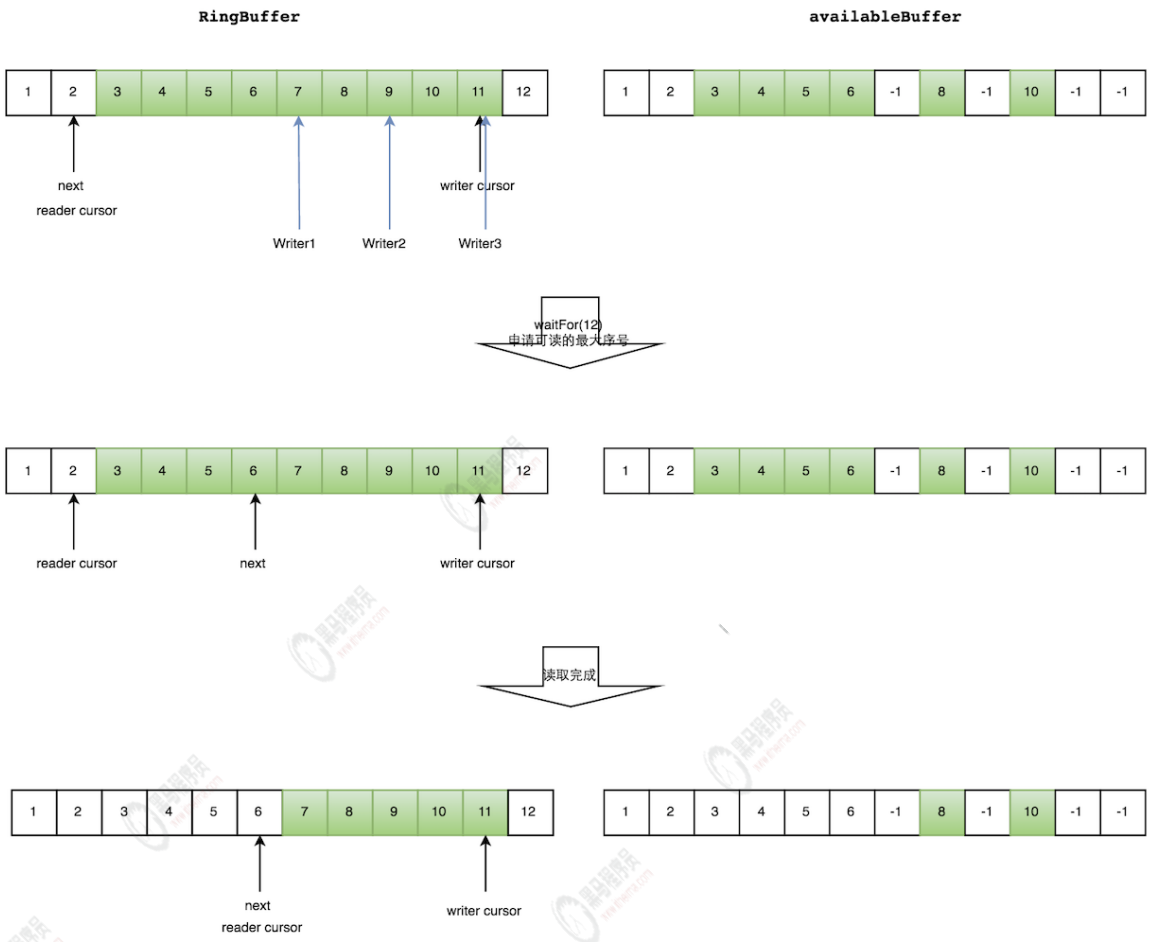
> 绿色代表已经写OK的数据

假设三个生产者在写中，还没有置位AvailableBuffer，那么消费者可获取的消费下标只能获取到6，然后等生产者都写OK后，通知到消费者，消费者继续重复上面的步骤。

消费流程

1. 申请读取到序号n；
2. 若writer cursor >= n，这时仍然无法确定连续可读的最大下标。从reader cursor开始读取available Buffer，一直查到第一个不可用的元素，然后返回最大连续可读元素的位置；
3. 消费者读取元素

> 如下图所示，读线程读到下标为2的元素，三个线程Writer1/Writer2/Writer3正在向RingBuffer相应位置写数据，写线程被分配到的最大元素下标是11。
>
> 读线程申请读取到下标从3到11的元素，判断writer cursor>=11。然后开始读取availableBuffer，从3开始，往后读取，发现下标为7的元素没有生产成功，于是WaitFor(11)返回6。
>
> 然后，消费者读取下标从3到6共计4个元素。

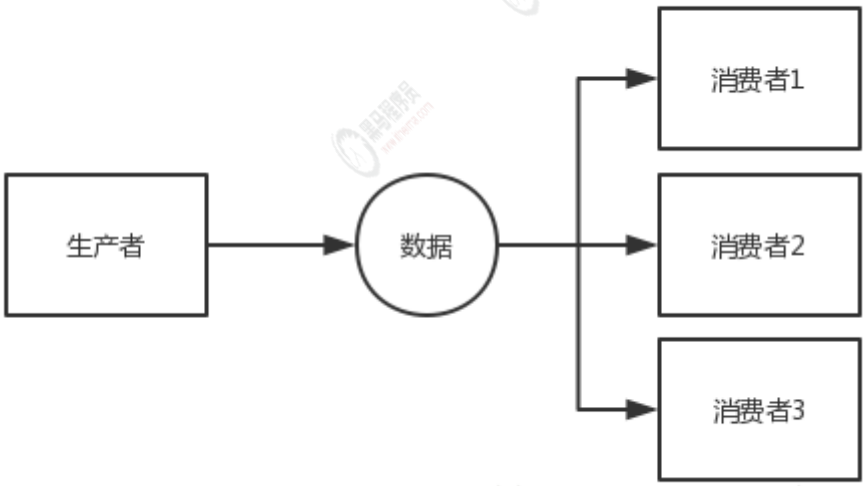


高级使用

并行模式

单一写者模式

在并发系统中提高性能最好的方式之一就是单一写者原则，对Disruptor也是适用的。如果在你的代码中仅仅有一个事件生产者，那么可以设置为单一生产者模式来提高系统的性能。



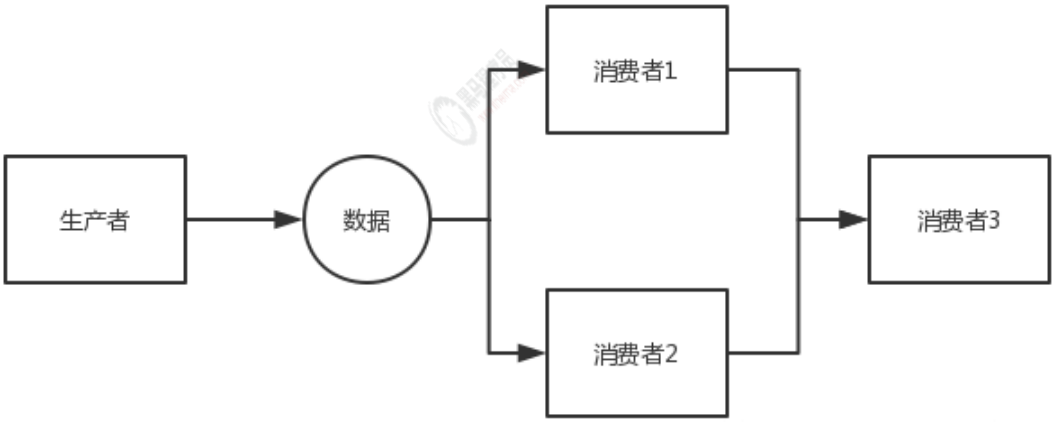
```
public class singleProductorLongEventMain {
    public static void main(String[] args) throws Exception {
        //.....// Construct the Disruptor with a SingleProducerSequencer

        Disruptor<LongEvent> disruptor = new Disruptor(factory,
                bufferSize,
                ProducerType.SINGLE, // 单一写者模式,
                executor);//.....
    }
}
```

串行消费

> 比如：现在触发一个注册Event，需要有一个Handler来存储信息，一个Hanlder来发邮件等等。

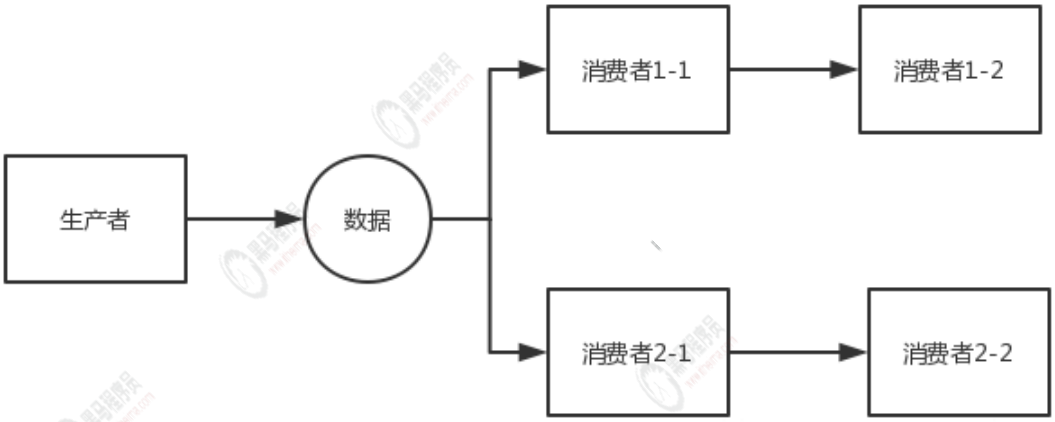
```
/**
  * 串行依次执行
  * <br/>
  * p --> c11 --> c21
  * @param disruptor
  */
 public static void serial(Disruptor<LongEvent> disruptor){
     disruptor.handleEventsWith(new C11EventHandler()).then(new
C21EventHandler());
     disruptor.start();
 }
```

菱形方式执行



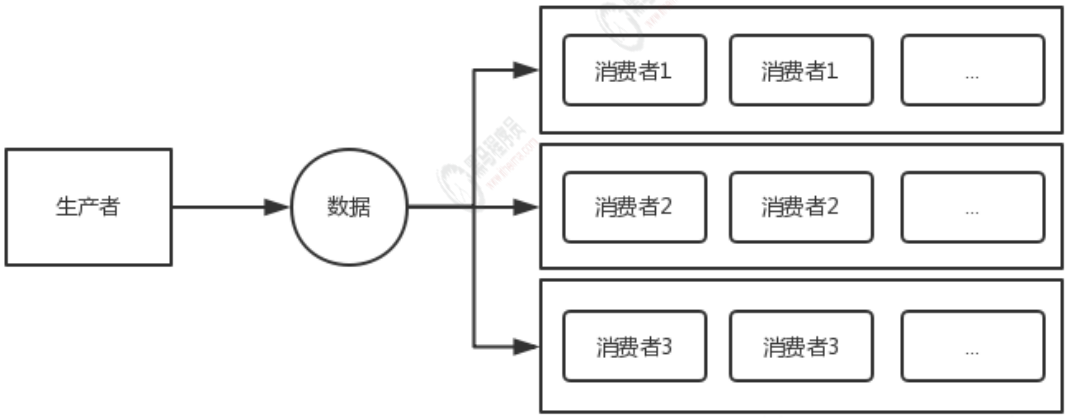
```
 public static void diamond(Disruptor<LongEvent> disruptor){
     disruptor.handleEventsWith(new C11EventHandler(),new
C12EventHandler()).then(new C21EventHandler());
     disruptor.start();
 }
```

链式并行计算



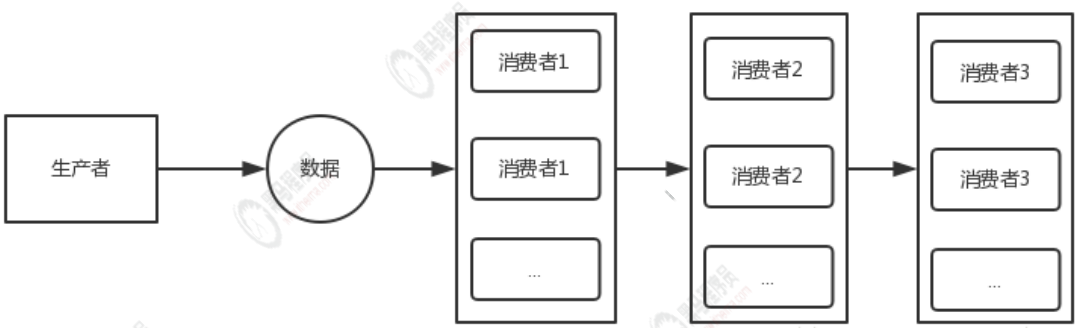

```
 public static void chain(Disruptor<LongEvent> disruptor){
     disruptor.handleEventsWith(new C11EventHandler()).then(new
C12EventHandler());
     disruptor.handleEventsWith(new C21EventHandler()).then(new
C22EventHandler());
     disruptor.start();
 }
```

相互隔离模式



```
 public static void parallelWithPool(Disruptor<LongEvent> disruptor){
     disruptor.handleEventsWithWorkerPool(new C11EventHandler(),new
C11EventHandler());
     disruptor.handleEventsWithWorkerPool(new C21EventHandler(),new
C21EventHandler());
     disruptor.start();
 }
```

航道模式




```
/**
  * 串行依次执行,同时C11，C21分别有2个实例
   * <br/>
   * p --> c11 --> c21
   * @param disruptor
   */
  public static void serialWithPool(Disruptor<LongEvent> disruptor){
      disruptor.handleEventsWithWorkerPool(new C11EventHandler(),new
C11EventHandler()).then(new C21EventHandler(),new C21EventHandler());
      disruptor.start();
  }
```